

Uni[MASK]: One Masked-Prediction Model for Every Decision-Making Task

NeurIPS 2022. UC Berkeley, Microsoft Research, and Carnegie Mellon recast behavior cloning, offline RL, dynamics, and goal conditioning as a single masking problem.

The recipe behind BERT is almost embarrassingly simple: hide some tokens in a sequence at random, ask a bidirectional model to fill them back in, and you get representations that transfer to a whole family of downstream tasks. That single idea reorganized much of natural language processing. This paper asks a pointed question. If trajectories in decision making are also just sequences, does the same trick reorganize how we build models for them?

In reinforcement learning and imitation learning, a trajectory is a sequence of states, actions, and returns. Yet we usually train a separate model for each question we want to ask of that sequence: one network for behavior cloning, another for offline RL and reward conditioning, another for forward or inverse dynamics, another for goal or waypoint conditioning. Every one of these models reads the same underlying object, and every one of them is built and trained in isolation, throwing away the structure the tasks share.

Uni[MASK] starts from a simple observation: each of these questions is just a different mask over one trajectory, revealing some tokens and predicting the rest. Written that way, the tasks stop being different problems and become different queries to the same model. A single bidirectional transformer, trained across many masking patterns, can answer all of them, and as the experiments show, it often matches or beats the specialized single-task models it is meant to replace, and consistently beats them after a short round of fine-tuning.

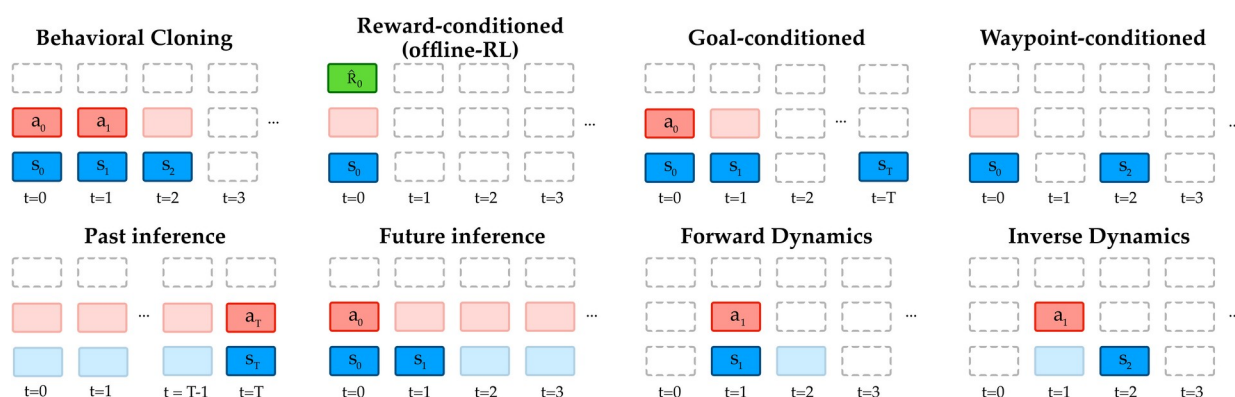


Figure 1. Familiar decision-making tasks written as masking schemes over one trajectory. Solid tokens are handed to the model; translucent tokens are what it must predict. Behavior cloning, reward and goal conditioning, waypoint conditioning, and forward and inverse dynamics differ only in which state, action, and return tokens are masked.

Paper: <https://arxiv.org/abs/2211.10869> · Code: <https://github.com/micahcarroll/uniMASK>

A model zoo for a single trajectory

Consider everything a practitioner might want from a batch of logged trajectories. Behavior cloning asks: given the states and actions seen so far, what action comes next? Offline RL, in its reward-conditioned form, asks for an action that will hit a target return. Forward dynamics predicts the next state from the current state and action; inverse dynamics recovers the action that connected two states. Goal and waypoint conditioning ask for behavior that passes through specified future states. These read like different research areas, each with its own architectures and training pipelines, yet all of them operate on one and the same trajectory of states, actions, and returns.

The cost of treating them separately is not just engineering overhead. A model trained only for behavior cloning never learns what a good dynamics model would teach it about the environment, and a dynamics model never benefits from the behavioral structure that goal conditioning captures. If the tasks genuinely share representations, training them in isolation leaves value on the table.

Every task is just a masking scheme

Uni[MASK] tokenizes a trajectory one timestep at a time. Each timestep contributes a state token, an action token, and optionally a property token such as return-to-go, the sum of rewards from that step onward. A masking scheme then specifies two things independently: which tokens the model gets to see as input, and which masked tokens it must predict and be scored on. Different schemes recover different tasks. Reveal the past states and actions and predict the next action, and you have behavior cloning. Additionally reveal a future state, and you have goal conditioning. Reveal the return-to-go token, and you have reward-conditioned offline RL. Nothing about the model changes between these; only the mask does.

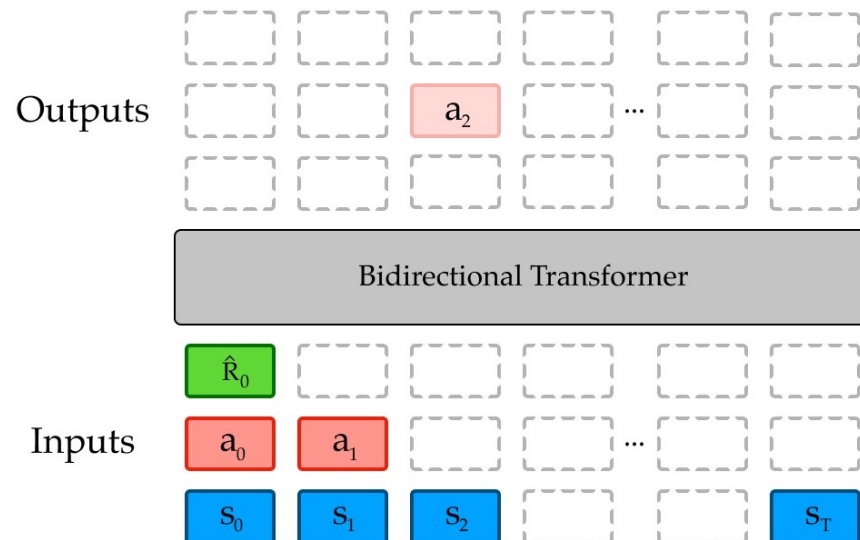


Figure 2. The backbone is a BERT-style bidirectional transformer. Each timestep's state, action, and property tokens are stacked into one input; the model reads all visible tokens at once and predicts the masked outputs, here the next action given past states, past actions, and the initial return-to-go.

Because the encoder is bidirectional, it can condition on tokens from anywhere in the window, past or future, which is exactly what tasks like goal conditioning and backward inference need. This is the crucial departure from a left-to-right GPT: a strictly causal model that only conditions on the past, and it is what lets one network express the full family of inferences instead of only the causal ones.

Four ways to train one model

Having a single model that can express every task still leaves the question of how to train it. The paper compares four regimes. Single-task trains one model on one masking scheme, the classic baseline. Multi-task samples, for each trajectory snippet, one of the predefined task masks. Random-mask goes further and masks arbitrary tokens, never committing to a named task at all. Finetune first pretrains with random masking, then fine-tunes on a specific task. The comparison is designed so that each regime isolates the effect of one ingredient, with fine-tuning as the one to watch.

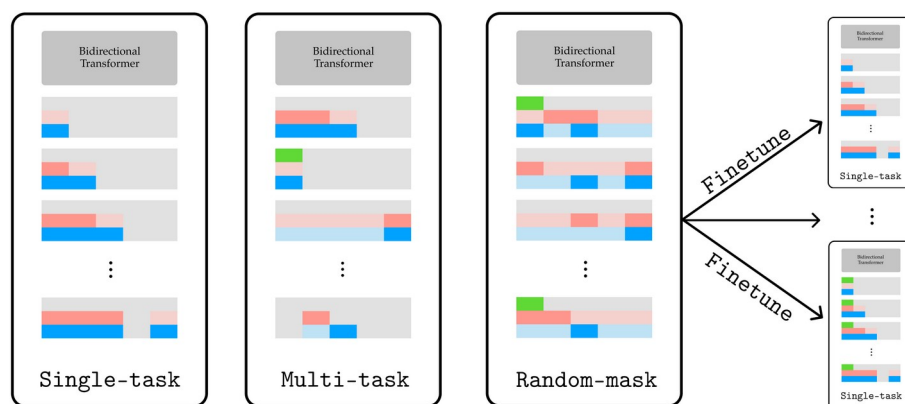


Figure 3. The four training regimes. Single-task and multi-task train on named task masks; random-mask trains on arbitrary token masks; the fine-tune regime pretrains with random masking and then specializes to a single task.

One model, many behaviors

To make the framework tangible, the authors turn to MiniGrid, a small gridworld where an agent must pick up a key, open a locked door, and reach a goal. A single model trained with random masking is then queried with different masks, and each mask draws out a different behavior, all from the same weights.

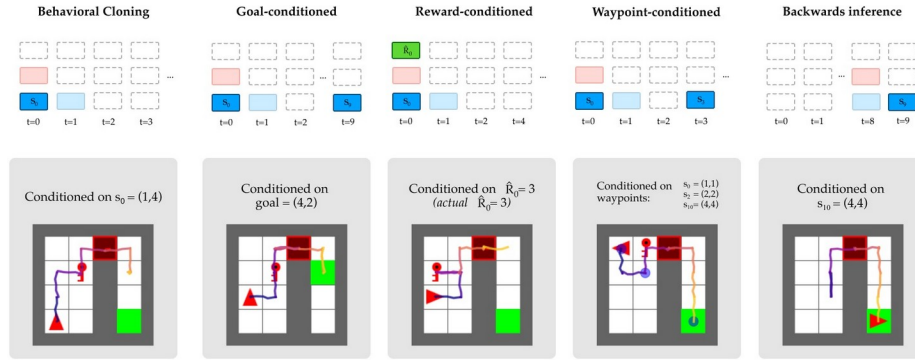


Figure 4. One random-mask model, five different queries. From left to right: behavior cloning from an initial state, reaching an alternative goal, hitting a target return, threading specified waypoints, and backward inference of a plausible history that ends at a given final state. Trajectories are drawn with jitter for clarity.

This is the qualitative payoff of the reframing. Reaching an alternative goal, obeying waypoints, or reconstructing a plausible past are not separate models bolted together; they are the same model answering different masked queries. The framework even extends to arbitrary property labels, for example conditioning on whether a trajectory ever visits a particular corner of the grid.

What the numbers say

The qualitative demo is convincing, but the quantitative comparison is where the reframing has to earn its keep. On MiniGrid, the authors measure task-specific validation losses, averaged over 6 seeds and normalized so that the best method in each column equals 1. Two patterns stand out. Training with random masking beats single-task training on every task, so a single generalist outperforms the specialists. And random-mask pretraining followed by fine-tuning is best of all, beating single-task models on every task except behavior cloning.

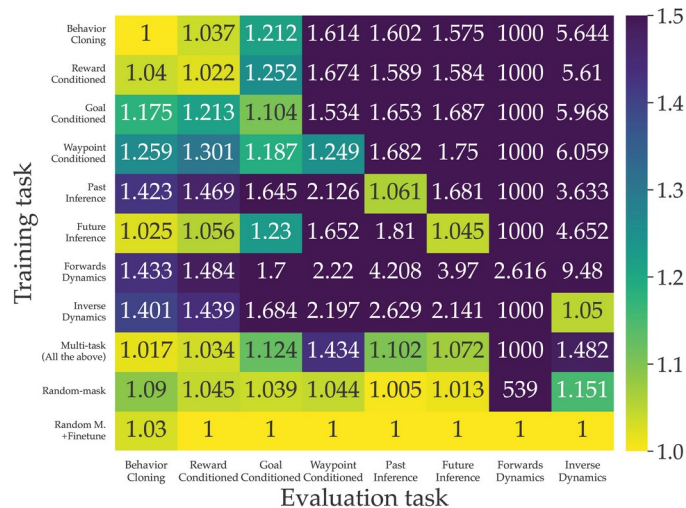


Figure 5. Task-specific validation losses on MiniGrid, normalized column-wise so the best method in each evaluation task equals 1; lower is better. The bottom row, random-mask pretraining plus per-task fine-tuning, is best or tied-best across the board, while the single-task rows near the top generalize poorly to tasks they were not trained on.

The harder test is Maze2D, a continuous-control maze from the D4RL benchmark, built on MuJoCo physics, evaluated by test-time reward over 1,000 rollouts across 5 seeds. Here fine-tuning is not just helpful, it is decisive. At a context length of 5, the fine-tuned Uni[MASK] models reach a reward of about 2.73 on both behavior cloning and reward conditioning, against 1.13 and 1.49 for a Decision Transformer, and lower still for a plain feedforward network. Training broadly and then specializing beats training narrowly from the start.

Table 1. Maze2D test-time reward at context length 5 (higher is better).

Method	Behavior cloning	Reward conditioning
Uni[MASK] (pretrain + fine-tune)	2.73	2.73
Decision Transformer	1.13	1.49

Rewards are averaged over 5 seeds and 1,000 rollouts.

The comparison also exposes an honest limitation. To isolate the backbone, the authors pit single-task Uni[MASK] against Decision-GPT, an improved GPT-style baseline they introduce, where the only real difference is a BERT-style versus a GPT-style architecture. At a context length of 5 the BERT-style model does well, but at a context length of 10 it degrades and is overtaken by Decision-GPT, a known difficulty of BERT-like models with longer-sequence generation.

Takeaway

The lasting idea is clean. Many tasks that the field treats as distinct, including behavior cloning, offline RL, dynamics, and goal and waypoint conditioning, are just different maskings of the same trajectory, so one masked-prediction model can stand in for a zoo of specialized ones. And training across many masks, then fine-tuning, generally beats training on any single task alone, even when that single task is all you care about.

The caveat is architectural rather than conceptual. BERT-style bidirectional models shine at short context lengths but struggle to generate over longer sequences, which is where the GPT-style Decision-GPT pulls ahead. That points to a natural next step the authors call out: pairing a GPT-style backbone with random masking, to keep the unification while recovering strong long-horizon generation.